

Warm up

No technical warm up question, but talk to your neighbours! Some things you can talk about:

- How was the midterm?
- Spring break plans
- Favourite data structure
- Favourite data type in C
- Favourite type of cloud

Precept 11: Making abstract data types generic

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

March 5, 2026

Overview

- 1 `void` pointers
- 2 A generic stack abstract data type
- 3 Assignment 3: a symbol table ADT
- 4 Summary

void

- `void` is a special type that represents the absence of a value or type
- Commonly used in function declarations to indicate that a function does not return a value or does not take any parameters
 - `void exit(int status);`
 - `int main(void);`
- Cannot create a variable of type `void`, i.e., `void x;` is not valid
- Also used in pointer types to indicate that the pointer can point to any data type (i.e., `void*`)

What is a `void` pointer?

- A `void` pointer (i.e., `void*`) is a special type of pointer that can point to any data type
- Used in C to create generic data structures and functions that can work with different types of data without needing to know the specific type at compile time

- `void* malloc(size_t size);`
 - `void free(void* ptr);`
 - `void qsort(void* base, ...);`

- But: cannot directly dereference it or perform pointer arithmetic on it without first casting it to the appropriate type
 - Otherwise: how much memory to dereference? how to interpret the bytes at that memory location?
 - Example:

```
int x = 42;
void* px = &x;
*(int*)px = 67; /* cast before dereferencing */
```

Warning: `void` pointers and type safety

- `void` pointers subvert the type system → can lead to type safety issues if not used carefully
- If you cast a `void` pointer to the wrong type, you may end up with undefined behaviour when you try to dereference it

– Example:

```
int x = 42;  
void* px = &x;  
*(double*)px = 3.14; /* undefined behaviour */
```

- Always ensure that you are casting to the correct type before dereferencing a `void` pointer, and be mindful of the potential for type safety issues when using `void` pointers in your code
- More generally, avoid `void` pointers when possible, unless you have a good reason to use them (i.e., you don't know the proper data type)

Overview

- 1 void pointers
- 2 A generic stack abstract data type
- 3 Assignment 3: a symbol table ADT
- 4 Summary

Last time: a stack ADT with `doubles` (stack4)

```
/* stack.h */
typedef struct Stack *Stack_T;

/* stack.c */
#include "stack.h"

struct StackNode {
    double dItem;           /* the item */
    struct StackNode *psNextNode; /* next StackNode */
};

struct Stack {
    struct StackNode *psFirstNode;
};
```

- Downside: can only store `doubles` in this stack
- What if we want to store other types of data?
 - Solution: store `void` pointers instead of `doubles`

A generic stack ADT with `void` pointers (stack5)

```
/* stack.h */
typedef struct Stack *Stack_T;

/* stack.c */
#include "stack.h"

struct StackNode {
    const void *pvItem;           /* the item */
    struct StackNode *psNextNode; /* next StackNode */
};

struct Stack {
    struct StackNode *psFirstNode;
};
```

- Why make `pvItem` a `const void*` instead of just a `void*`?
 - Promise by the stack interface to not modify items stored in the stack
 - Caller still “owns” the item and is responsible for managing its memory (e.g., allocating and deallocating it)

Implementing and using the generic stack ADT

- In function implementations, replace `double` with `const void*`, e.g., `int Stack_push(Stack_T oStack, const void *pvItem);`
- To use the stack, we need to create the item we want to store and pass a pointer to it
 - Should the item be dynamically allocated or can it be stack allocated?
 - ◇ It depends!
 - Stack-based allocation is simpler and more efficient, but the item will only be valid within the scope of the function where it is created
 - Heap-based allocation allows the item to persist beyond the scope of the function (more manual memory management, but less likely to run into dangling pointer issues)
 - ◇ This is what `testclient.c` does

Overview

- 1 void pointers
- 2 A generic stack abstract data type
- 3 Assignment 3: a symbol table ADT
- 4 Summary

Symbol table ADT specifications

A symbol table is an unordered collection of bindings. A binding consists of a key and a value.

- `SymTable_T SymTable_new(void);`
- `void SymTable_free(SymTable_T oSymTable);`
- `size_t SymTable_getLength(SymTable_T oSymTable);`
- `int SymTable_put(SymTable_T oSymTable, const char *pcKey, const void *pvValue);`
- `void *SymTable_replace(SymTable_T oSymTable, const char *pcKey, const void *pvValue);`
- `int SymTable_contains(SymTable_T oSymTable, const char *pcKey);`
- `void *SymTable_get(SymTable_T oSymTable, const char *pcKey);`
- `void *SymTable_remove(SymTable_T oSymTable, const char *pcKey);`
- `void SymTable_map(SymTable_T oSymTable, void (*pfApply)(const char *pcKey, void *pvValue, void *pvExtra), const void *pvExtra);`

Overview

- 1 void pointers
- 2 A generic stack abstract data type
- 3 Assignment 3: a symbol table ADT
- 4 Summary

Summary

- The `void` type
- `void` pointers
- Using `void` pointers to create generic data structures
- Assignment 3 overview
- Dates to remember:
 - Have a great spring break :)
 - Assignment 3: due Tue, 3/24 at 9pm
 - Assignment 3 Quiz: Wed, 3/25 in lecture
- Acknowledgements
 - The stack implementations in these slides are adapted from the precept materials, written by Prof. Bob Dondero
 - Can be accessed at <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w06p1/>